

espol-cuboids
Polarizability Sweep Suite — User Guide

July 6, 2026

Contents

1	Introduction	2
2	Project Structure	2
3	Prerequisites	3
4	Running the MoM 3D Sweep	3
4.1	Command-Line Options	3
4.2	Shape Grid	3
4.3	Convergence	3
4.4	Examples	4
4.5	Output	4
5	Running the MoM 2D Sweep	4
5.1	Command-Line Options	4
5.2	Shape Grid	4
5.3	Convergence	4
5.4	Examples	4
5.5	Output	5
6	Running the DDA Sweep	5
6.1	Command-Line Options	5
6.2	Modes	5
6.3	Examples	5
6.4	Output	6
7	Interpreting Results	6
7.1	Depolarization Factors	6
7.2	Convergence Extrapolation	6
7.3	PEC Limit	6
8	Caching	6
9	Running Tests	7
10	Further Reading	7

1 Introduction

This suite computes the electrostatic polarizability tensor of dielectric cuboids using three independent numerical methods:

- **MoM 3D** — 3D surface integral equation (Method of Moments) for rectangular cuboids of arbitrary aspect ratio.
- **MoM 2D** — 2D boundary integral equation for infinitely long prisms with rectangular cross-section.
- **DDA** — Discrete Dipole Approximation (volume integral equation) on a cubic lattice inside the cuboid.

All three methods output the normalised polarizability tensor and effective depolarization factors N_x, N_y, N_z for each shape and permittivity. The MoM and DDA solvers also perform convergence extrapolation to the continuum limit.

For mathematical details of each method, refer to the accompanying implementation documents in this folder:

- `implementation_mom3d.md`
- `implementation_mom2d.md`
- `implementation_dda.md`

2 Project Structure

Listing 1: Key files and directories

```
espol-cuboids/
  run_mom3d_sweep.py      # 3D MoM sweep entry point
  run_mom2d_sweep.py      # 2D MoM sweep entry point
  run_dda_sweep.py        # DDA sweep entry point

  tools/
    gen_mom_3d.py          # 3D MoM: convergence, caching, power-law fit
    gen_mesh_3d_rect.py    # 3D surface mesh generator
    sm_comp_polarizability_3d.py # 3D MoM core solver
    mom3d_kernel.py        # Matrix-free GMRES operator
    mom3d_aca.py           # ACA-compressed H-matrix solver
    gen_mesh_2d_rect.py    # 2D perimeter mesh generator
    sm_comp_polarizability_2d.py # 2D MoM core solver
    vector_analysis.py      # V3, D3, Mesh data structures
    dda_core.py            # DDA: lattice, FFT-GMRES, convergence
    _path_setup.py         # Python path bootstrap

  docs/
    user_guide.tex         # This document
    implementation_mom3d.md
    implementation_mom2d.md
    implementation_dda.md

  tests/
    test_vector.py         # V3, D3, Mesh unit tests
    test_dda_core.py       # DDA correctness & regression
    test_mom2d.py          # 2D MoM mesh, solver, regression
```

```

test_mom3d.py          # 3D MoM mesh, solver, ACA, convergence

data/csvdata/          # Output CSV files
data/_mom3d_cache/     # MoM 3D solve cache
data/_dda_cache/       # DDA solve cache

```

3 Prerequisites

- Python 3.10+ with NumPy, SciPy.
- All scripts assume they are run from the `espol-cuboids/` directory.
- The `tools/` folder is automatically added to the Python path by each entry script.
- Cached results are stored in `data/_mom3d_cache/` and `data/_dda_cache/`. Delete these directories to force recomputation.

4 Running the MoM 3D Sweep

4.1 Command-Line Options

```
python run_mom3d_sweep.py [--er ER_LIST] [--solver direct|aca] [--dry-run]
```

Flag	Values	Description
<code>--er</code>	Comma-separated floats	Relative permittivities to sweep. Default: <code>10,20,100,1e10</code> (where $1e10 \approx \text{PEC}$).
<code>--solver</code>	<code>direct</code> (default), <code>aca</code>	Solver backend. <code>direct</code> uses LU factorization; <code>aca</code> uses ACA-compressed H-matrix + GMRES (recommended for large meshes, $N > 2000$).
<code>--dry-run</code>	(no value)	Print the shapes, ε_r values, and n_d sequences that would be computed, then exit.

4.2 Shape Grid

The sweep covers 15 aspect ratios $(u, v) = (l_y/l_x, l_z/l_x)$:

`(1.0, 1.0), (1.0, 0.7), (1.0, 0.5), (1.0, 0.3), (1.0, 0.2), (0.7, 0.7),`
`(0.7, 0.5), (0.7, 0.3), (0.7, 0.2), (0.5, 0.5), (0.5, 0.3), (0.5, 0.2),`
`(0.3, 0.3), (0.3, 0.2), (0.2, 0.2)`

with $l_x = 1$ fixed.

4.3 Convergence

For each shape, the solver runs 4–6 mesh resolutions (n_d) chosen adaptively by `adaptive_nd()`. A power-law fit $P(n_d) = P_\infty + C n_d^{-\beta}$ extrapolates to the continuum limit. Individual solves and converged fits are cached on disk.

4.4 Examples

```
# Preview what will be computed
python run_mom3d_sweep.py --dry-run

# Sweep with default permittivities (direct LU)
python run_mom3d_sweep.py

# Single permittivity with ACA solver
python run_mom3d_sweep.py --er 10 --solver aca

# Custom permittivity range
python run_mom3d_sweep.py --er 1.5,3,6,12,100
```

4.5 Output

Results are printed to stdout as the sweep progresses. The depolarization factors N_x, N_y, N_z and their sum $N_\Sigma = N_x + N_y + N_z$ are shown for each shape- ϵ_r pair.

For bulk data export, use the standalone `gen_mom_3d.main()` function (run `python tools/gen_mom_3d.py --dry-run`).

5 Running the MoM 2D Sweep

5.1 Command-Line Options

```
python run_mom2d_sweep.py [--er ER_LIST] [--dry-run]
```

Flag	Values	Description
<code>--er</code>	Comma-separated floats	Relative permittivities. Default: 10,20,100,1e10.
<code>--dry-run</code>	(no value)	Print the shapes and n_d sequences that would be computed, then exit.

The 2D solver uses only the direct LU backend (no ACA/GMRES option), as problem sizes remain small ($n \lesssim 500$).

5.2 Shape Grid

11 aspect ratios $r = l_y/l_x$ with $l_x = 1$:

1.0, 0.7, 0.5, 0.3, 0.2, 0.15, 0.1, 0.07, 0.05, 0.03, 0.02

5.3 Convergence

n_d sequences are chosen adaptively. A power-law fit extrapolates to $n_d \rightarrow \infty$. Individual solves and fits are cached in `data/_mom2d_cache/`.

5.4 Examples

```
# Preview
python run_mom2d_sweep.py --dry-run

# Full sweep
python run_mom2d_sweep.py

# Custom permittivities
python run_mom2d_sweep.py --er 2,5,10
```

5.5 Output

For each shape- ϵ_r pair, the normalized depolarization factors

$$N_x = \frac{1}{P_{xx}/A}, \quad N_y = \frac{1}{P_{yy}/A}, \quad N_\Sigma = N_x + N_y$$

are printed to stdout. Results are saved to `data/csvdata/mom2d.csv`.

6 Running the DDA Sweep

6.1 Command-Line Options

```
python run_dda_sweep.py --mode fixed|converge [--er ER_LIST] [--dry-run]
```

Flag	Values	Description
--mode	fixed, converge	fixed : single lattice spacing $d = s_{\min}/10$. converge : multiple d levels with $d \rightarrow 0$ fit.
--er	Comma-separated floats	Relative permittivities. Default: 0.5, 1.25, 1.5, 1.75, 2, 3, 5, 8, 10, 12.5.
--dry-run	(no value)	Print the shapes and lattice parameters, then exit.

The DDA sweep shares the same 15-shape grid as the MoM 3D sweep (Section 4.2). The FFT-accelerated GMRES backend is used for all solves ($N \gtrsim 1000$).

6.2 Modes

fixed Each shape is solved at a single lattice spacing $d = \min(l_y, l_z)/10$. Fast, suitable for quick surveys. Output: `data/csvdata/dda3d.csv`.

converge Each shape is solved at 4–5 values of d (chosen so that $N \leq 20\,000$ dipoles per solve).

A power-law $P(d) = P_\infty + C d^\beta$ fit yields the continuum limit. Output: `data/csvdata/dda_convergence.csv` (individual solves) and `data/csvdata/dda_convergence_cnv.csv` (extrapolated values).

6.3 Examples

```
# Preview fixed mode
python run_dda_sweep.py --mode fixed --dry-run

# Quick fixed-d sweep (all er, all shapes)
python run_dda_sweep.py --mode fixed
```

```
# Converge mode for er=10 only
python run_dda_sweep.py --mode converge --er 10

# Converge mode with multiple er values
python run_dda_sweep.py --mode converge --er 2,5,10
```

6.4 Output

Depolarization factors N_x, N_y, N_z and their sum are printed per shape- ε_r - d combination. CSV files are written to the `data/csvdata/` directory.

7 Interpreting Results

7.1 Depolarization Factors

The depolarization factor N_i (for axis i) relates the internal field to the applied field in an ellipsoid:

$$E_i^{\text{int}} = E_i^{\text{app}} - \frac{N_i}{\varepsilon_0} P_i$$

For cuboids, N_i is shape- and permittivity-dependent (unlike ellipsoids, where it depends only on shape). The sum rule

$$N_x + N_y + N_z = 1 \quad (3\text{D}), \quad N_x + N_y \approx 1 \quad (2\text{D})$$

holds exactly for ellipsoids and approximately for cuboids at moderate ε_r .

7.2 Convergence Extrapolation

The power-law fit $P(d) = P_\infty + C d^\beta$ is justified by the leading discretization-error term for smooth geometries. The exponent β is typically 1–2 for the MoM and 1–3 for the DDA. Fits are monitored; if the residual is large or β falls outside expected bounds, the finest-resolution value is used as a fallback.

7.3 PEC Limit

For the DDA, the PEC limit is obtained by fitting $\alpha(\varepsilon_r) = \alpha_{\text{PEC}} + A/\varepsilon_r$. For the MoM, $\varepsilon_r = 10^{10}$ is used as a proxy (the system matrix becomes nearly singular for true PEC, but $\varepsilon_r = 10^{10}$ gives the correct limit to ~ 4 significant digits).

8 Caching

All three sweeps cache individual solves on disk:

- **MoM 3D:** `data/_mom3d_cache/` — keyed by ε_r , shape, n_d , β , and solver type (direct/ACA).
- **MoM 2D:** `data/_mom2d_cache/` — keyed by ε_r , shape, n_d , β .
- **DDA:** `data/_dda_cache/` — keyed by ε_r , shape, and lattice spacing d .

Cache keys include version tags; changing the implementation version automatically invalidates stale cache entries. To force a full recomputation, delete the relevant cache directory.

9 Running Tests

The test suite is run with `pytest` from the `espol-cuboids/` directory:

```
python -m pytest tests/ -q
```

This runs all 54 tests covering vector operations, mesh generation, MoM 2D/3D solvers, DDA solver, ACA solver, convergence extrapolation, and regression against known values.

To skip slow tests (full MoM/DDA solves), use:

```
python -m pytest tests/ -q -m "not slow"
```

10 Further Reading

- `implementation_mom3d.md` — Detailed description of the 3D MoM formulation, octant symmetry, ACA compression, and power-law fit.
- `implementation_mom2d.md` — 2D PMCHW integral equation and collocation discretization.
- `implementation_dda.md` — Clausius–Mossotti polarizability, coupled-dipole equations, FFT-accelerated GMRES, and $d \rightarrow 0$ / $\varepsilon_r \rightarrow \infty$ extrapolation.